

AIMS System - Detailed Class Design Documentation

1. Design Class: Product

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	int	null	Unique Identifier for the product
2	title	String	null	Name of the product
3	category	String	null	Product category (Book, CD, DVD, or LP record)
4	value	float	0.0	Base price of the product before VAT
5	price	float	0.0	Selling price of the product
6	quantity	int	0	Available quantity in stock
7	description	String	null	Product description
8	imageUrl	String	null	The URL for the image of the product item
9	barCode	String	null	Unique barcode for the product
10	warehouseEntryDate	Date	null	Date when the product entered the warehouse
11	dimensions	String	null	Physical dimensions of the product (e.g., 10x20x5 cm)
12	weight	float	null	Weight of the product in kilograms
13	warehouseProvince	String	null	province of warehouse storing the product
14	warehouseDistrict	String	null	district of warehouse storing the product
15	warehouseAddress	String	null	detailed address of warehouse storing the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	checkProductAvailability	boolean	Checks if the product has sufficient quantity available
2	updateProduct	vold	Updates product information in the system
3	checkEligibilityOfRushDelivery	boolean	Checks if product is eligible for rush delivery
4	createProduct	vold	Create a new product in the system
5	searchProduct	vold	Search for a specific product attributes

Parameter:

- checkProductAvailability(quantity: int): Boolean
 - **quantity:** An integer representing the number of products to check if the available inventory is sufficient.
 - **Return:** true if the available inventory is sufficient, otherwise false.
- updateProduct(productId: int): vold
 - **productId:** productId must exist
- searchProduct(searchCriteria: Map<String, Object>): vold
 - **searchCriteria:** A map containing search criteria (field names and values)

Exception:

- updateProduct(productId: int): vold
 - ProductNotFoundException: If no product with the specified productId is found.
 - ExceedUpdateTimesException: If update more than 30 products a day
 - ExceedUpdatePriceTimesPerDayException: If update price more than twice a day

Method:

How to use parameters/attributes:

- checkProductAvailability(quantity: int): Boolean
 - **Purpose:** This method checks whether the available inventory is sufficient for the requested quantity of a product.
 - **Usage:** If the available stock is at least equal to the requested quantity, the method returns true; otherwise, it returns false.
- updateProduct(productId: int): void
 - **Purpose:** This method updates the details of an existing product based on the provided information.
 - **Usage:** If the product exists, the method updates attributes such as name, price, quantity, etc
- createProduct(): void
 - **Purpose:** This method creates a new product with the provided information.
 - **Usage:** If all required attributes are provided and valid, the product is successfully created and added to the inventory.
- checkEligibilityOfRushDelivery(address: String): Boolean
 - **Purpose:** to check rush delivery support
 - **Usage:** compare 'address' parameter (from delivery information) and address of product (based on information provided by Product Manager)
- searchProduct(searchCriteria: Map<String, Object>): void
 - **Purpose:** Searches for products matching specific criteria.
 - **Usage:** Used to find products based on various attributes.
 - **Processing:**
 1. Constructs a database query based on the search criteria
 2. Executes the query
 3. Returns matching products

State:

- If stock > requested quantity → Available.
- If stock < requested quantity → Not Available.
- If product_Id is invalid → ProductNotFoundException is raised.

2. Design Class: Book

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	author	String	null	Author of the book
2	coverType	String	null	Type of cover (paperback or hardcover)
3	publisher	String	null	Publisher of the book
4	publicationDate	Date	null	Date of publication
5	pages	int	0	Number of pages
6	language	String	null	Language of the book
7	genre	String	null	Genre of the book

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	void	Updates product information in the system
2	createProduct	void	Create a new product in the system

Parameter: None

Exception:

- updateProduct(productId: int): void
 - ExceedUpdatedTimesException: If update more than 30 products a day
 - ExceedUpdateTimesPerDayException: If update more than twice a day

Method:

How to use parameters/attributes:

- updateProduct(productId: int): void
 - **Purpose:** Updates attributes such as price, quantity, and description,.. of book.
 - **Usage:** If the book exists, the method updates attributes such as name, price, quantity, etc

- createProduct(): void
 - **Purpose:** Creates a new Book with specified attributes such as name, price, stock quantity, and description.
 - **Usage:** If all required attributes are provided and valld, the book is successfully created and added to the inventory.

3. Design Class: CD

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	artist	String	null	Name of the artist or band
2	recordLabel	String	null	Name of the record label
3	tracklist	String	null	List of tracks in the CD
4	genre	String	null	Music genre of the CD
5	releaseDate	Date	0	Official release date of the CD

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	void	Updates product information in the system
2	createProduct	void	Create a new product in the system

Parameter: None

Exception:

- updateProduct(productId: int): void
 - ExceedUpdatedTimesException: If update more than 30 products a day
 - ExceedUpdateTimesPerDayException: If update more than twice a day

Method:

How to use parameters/attributes:

- updateProduct(productId: int): void
 - **Purpose:** Updates attributes such as price, quantity, and description,.. of CD.
 - **Usage:** If the CD exists, the method updates attributes such as name, price, quantity, etc
- createProduct(): void
 - **Purpose:** Creates a new CD with specified attributes such as name, price, stock quantity, and description.
 - **Usage:** If all required attributes are provided and valld, the CD is successfully created and added to the inventory.

4. Design Class: DVD

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	discType	String	null	Type of disc (e.g: Blu-ray, HD-DVD)
2	director	String	null	Name of the artist
3	runtime	String	null	Total runtime of the DVD
4	studio	String	null	Name of the studio
5	language	String	null	Language of the content in DVD
5	subtitle	String	null	Language subtitle
6	releaseDate	Date	null	Release date of the DVD
7	genre	String	null	The genre of the DVD

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	void	Updates product information in the system

2	createProduct	vold	Create a new product in the system
---	---------------	------	------------------------------------

Parameter: None

Exception:

- updateProduct(productId: int): vold
 - ExceedUpdatedTimesException: If update more than 30 products a day
 - ExceedUpdateTimesPerDayException: If update more than twice a day

Method:

How to use parameters/attributes:

- createProduct():
 - **Purpose:** Creates a new DVD with specified attributes such as name, price, stock quantity, and description.
 - **Usage:** If all required attributes are provided and valld, the DVD is successfully created and added to the inventory.
- updateProduct(productId: int): vold
 - **Purpose:** Updates attributes such as price, quantity, and description,.. of DVD.
 - **Usage:** If the DVD exists, the method updates attributes such as name, price, quantity, etc

5. Design Class: LP record

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	artist	String	null	Name of the artist or band
2	recordLabel	String	null	Name of the record label
3	tracklist	String	null	List of tracks in the LP record
4	genre	String	null	Music genre of the LP record
5	releaseDate	Date	0	Official release date of the LP record

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	updateProduct	vold	Updates product information in the system
2	createProduct	vold	Create a new product in the system

Parameter: None

Exception:

- updateProduct(productId: int): vold
 - ExceedUpdatedTimesException: If update more than 30 products a day
 - ExceedUpdateTimesPerDayException: If update more than twice a day

Method:

How to use parameters/attributes:

- createProduct():
 - **Purpose:** Creates a new DVD with specified attributes such as name, price, stock quantity, and description.
 - **Usage:** If all required attributes are provided and valld, the DVD is successfully created and added to the inventory.
- updateProduct(productId: int): vold
 - **Purpose:** Updates attributes such as price, quantity, and description,.. of DVD.
 - **Usage:** If the DVD exists, the method updates attributes such as name, price, quantity, etc

6. Design Class: ProductController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1				

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	CreateProduct()	vold	Create a new product with provlded information
2	UpdateProduct(productId: int)	vold	Update product with updated information
3	ValldateProduct(productId: String)	boolean	Check the provlded information is valld or not

Parameter:

- **UpdateProduct(productId: int)**
 - productId must be exist
- **ValldateProduct(productId: int)**
 - productId must be exist
 - The input in the form must be valld

Exception:

- **UpdateProduct(productId: int)**
 - ProductNotFoundException: If no product with the specified productId is found.
- **ValldateProduct(productId: int)**
 - ProductNotFoundException: If no product with the specified productId is found.
 - WrongFormatInputException: If input is in wrong format
 - PriceOutOfRangeException: If input price is out of range 30%-150% value

Method:

How to use parameters/attributes:

- **CreateProduct(productId: int)**
 - **Purpose:** This method creates a new product with the provided information.
 - **Usage:** It receives product details from the user or system, then creates a new product and adds it to the product list.
- **UpdateProduct(productId: int)**
 - **Purpose:** This method updates the details of an existing product based on the provided information.
 - **Usage:** If the product exists, the method updates attributes such as name, price, quantity, etc
- **ValldateProduct(productId: int)**
 - **Purpose:** This method checks whether the provided product information is valld.
 - **Usage:** It verifies required fields such as title, price, category, and quantity. If the information is valld, the method returns true; otherwise, it returns false (e.g., price out of the allowed range, negative quantity).

State:

7. Design Class: History update

Table 1. Atrribute design

#	Name	Data type	Default value	Description
1	historyId	int	null	Unique Identifier for the product
2	time	Date	null	The time product manager update the product
3	isUpdatePrice	boolean	null	The price of the product is update or not
4	productId	int	null	The Id of the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	recordUpdate	vold	Records the update history of a product when changes are made
2	getUpdateHistory	List<HistoryUp date>	Retrieves the update history of a product based on productId.
3	getLastUpdate	HistoryUpdate	Retrieves the most recent update of a product.
4	deleteHistory	vold	Deletes the entire update history of a product based on productId.

Parameter:

- recordUpdate(productId: int, isUpdatePrice: boolean): void
 - **productId:** An integer representing the unique Identifier of the updated product.
 - **isUpdatePrice:** A boolean indicating whether the product price was updated.
 - **Return:** This method does not return a value. It records an update entry in the history log.
- getUpdateHistory(productId: int): List<HistoryUpdate>
 - **productId:** An integer representing the unique Identifier of the product.
 - **Return:** A list of HistoryUpdate objects containing all update records for the specified product. If no history exists, it returns an empty list.
- getLastUpdate(productId: int): HistoryUpdate
 - **productId:** An integer representing the unique Identifier of the product.
 - **Return:** A HistoryUpdate object containing the latest update details of the specified product. If no update history exists, it returns null or raises an exception.
- deleteHistory(productId: int): void
 - **productId:** An integer representing the unique Identifier of the product.
 - **Return:** This method does not return a value. It deletes all update history records related to the specified product.

Exception:

- recordUpdate(productId: int, isUpdatePrice: boolean): void
 - ProductNotFoundException: If no product with the specified product_Id is found.
- getUpdateHistory(productId: int): List<HistoryUpdate>
 - ProductNotFoundException: If no product with the specified product_Id is found.
- getLastUpdate(productId: int): HistoryUpdate
 - ProductNotFoundException: If no product with the specified product_Id is found.
- deleteHistory(productId: int): void
 - ProductNotFoundException: If no product with the specified product_Id is found.

Method:

How to use parameters/attributes:

- recordUpdate(productId: int, isUpdatePrice: boolean): void
 - Purpose: Records the update history of a product when changes are made.
 - Usage: When a product is updated (either information or price), this method creates a new record with time as the update timestamp and isUpdatePrice indicating whether the price was updated.
- getUpdateHistory(productId: int): List<HistoryUpdate>
 - Purpose: Retrieves the update history of a product based on productId.
 - Usage: If the product has an update history, the method returns a list of previous updates; otherwise, it may return an empty list.
- getLastUpdate(productId: int): HistoryUpdate
 - Purpose: Retrieves the most recent update of a product.
 - Usage: The method finds the latest update record of productId and returns it. If no history exists, it may return null or raise an error.
- deleteHistory(productId: int): void
 - Purpose: Deletes the entire update history of a product based on productId.
 - Usage: When the update history of a product needs to be removed (e.g., when the product is deleted from the system), this method deletes all related records.